

반도체장비AI비전실무

01_AI비전과 딥러닝



1. 강의 소개

컴퓨터 비전과 AI 비전

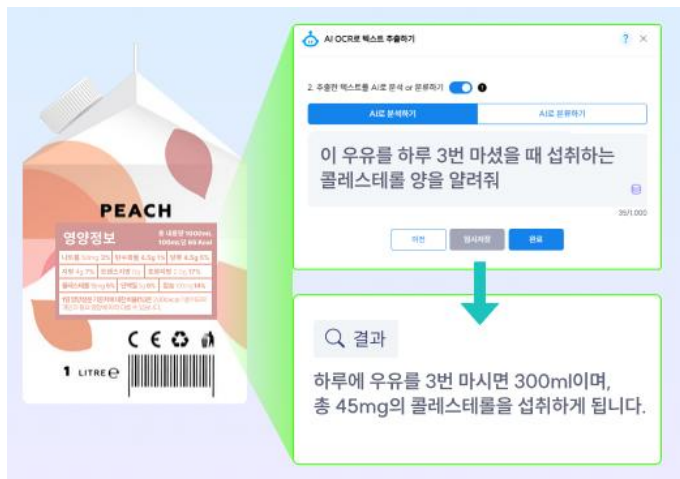
❖ 컴퓨터 비전(Computer Vision)

- 이미지에서 정보를 추출·인식·해석하는 기술

❖ AI 비전(AI Vision) $\subset$ 컴퓨터 비전

- 최신 인공지능과 딥러닝 기술을 활용해서
사물의 의미와 맥락까지 추론하는, 컴퓨터 비전의 더 발전된 형태
- 컴퓨터 비전이 인식 중심이라면, AI 비전은 이해/활용 중심

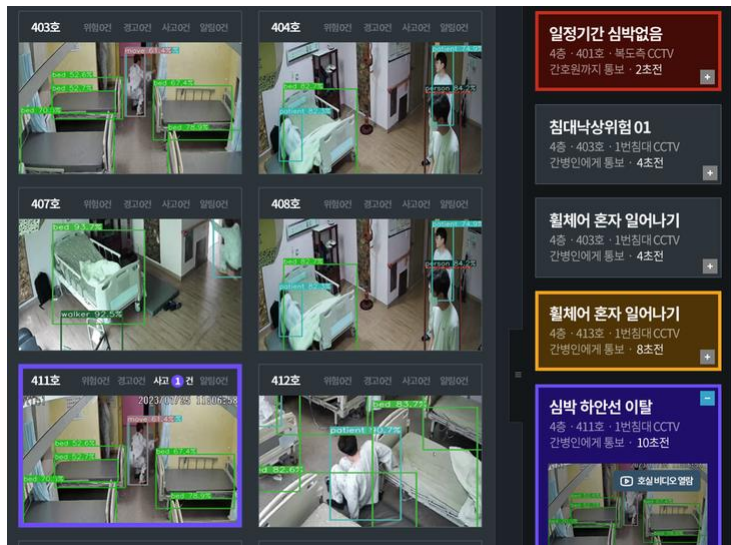
AI 비전 활용 사례



텍스트 인식: <https://blog.bati.ai/aiochr/>

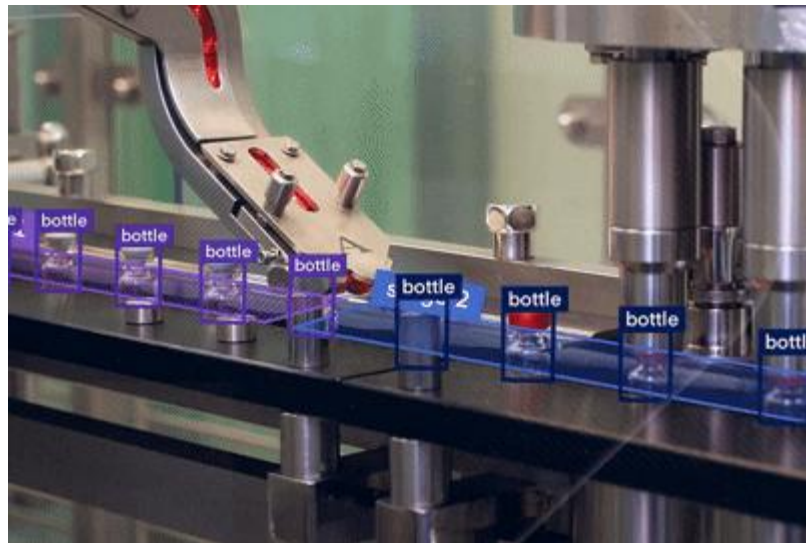
텍스트 인식: <https://store.cafe24.com/kr/story/4504>

AI 비전 활용 사례



안전사고예방 :

<https://www.ggilbo.com/news/articleView.html?idxno=1002463>

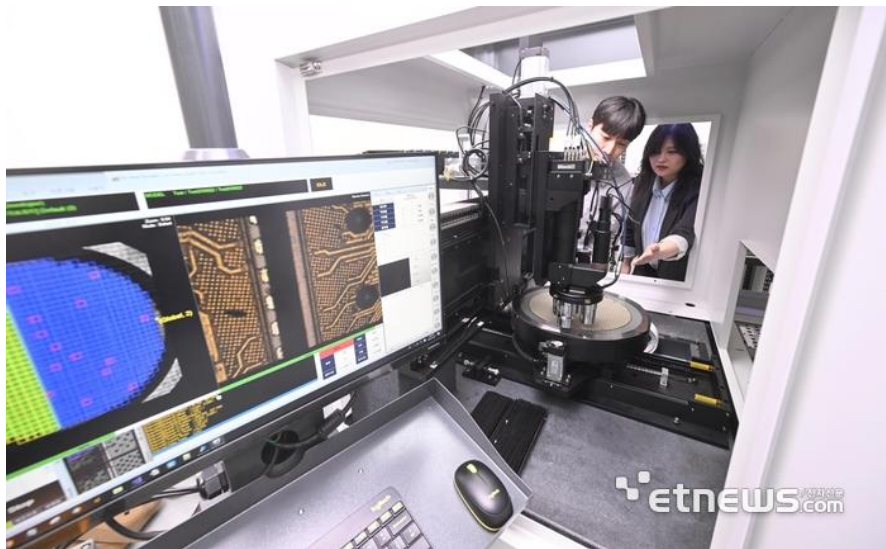
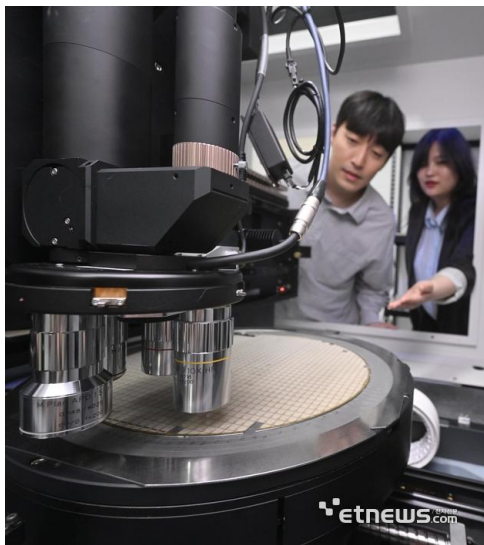


음료생산공정: <https://ai.movers.design/projects/factory-qaqc>

반도체장비AI비전이란?

❖반도체장비AI비전

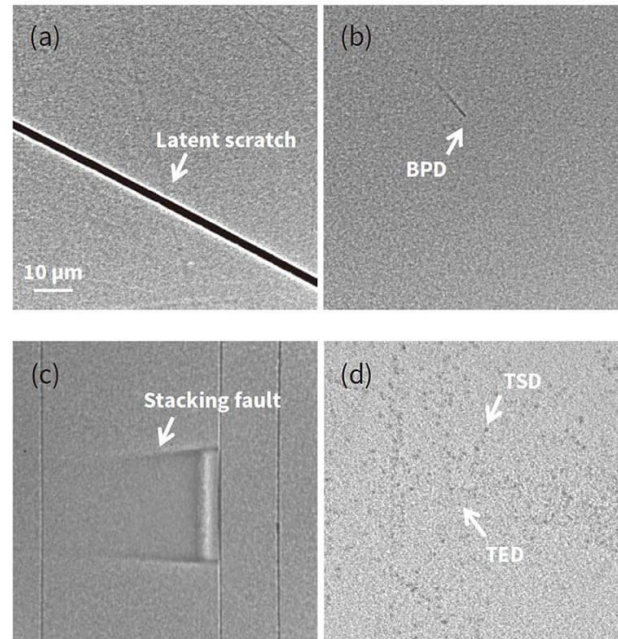
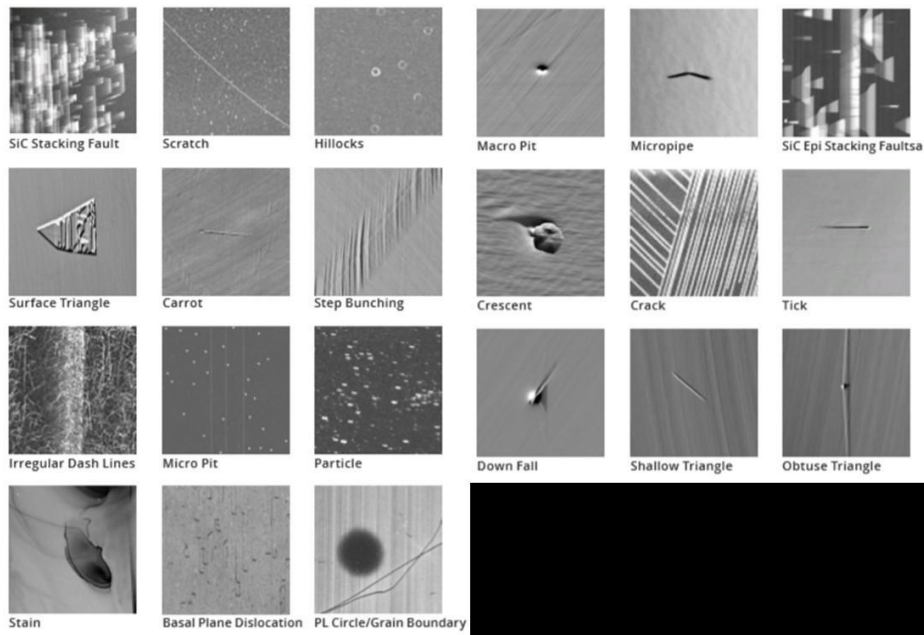
- 반도체공정 중에 발생한 이미지 데이터를 활용하는 AI비전
- 주로 수율 향상, 불량 검출 등에 활용된다.



반도체 검사 이미지의 종류

❖ 웨이퍼 표면 검사 이미지

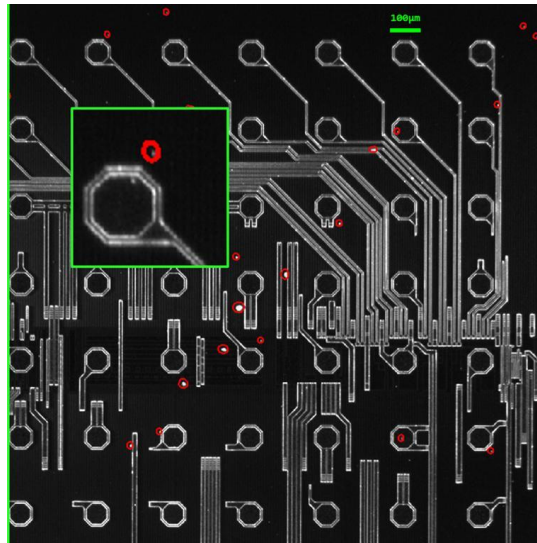
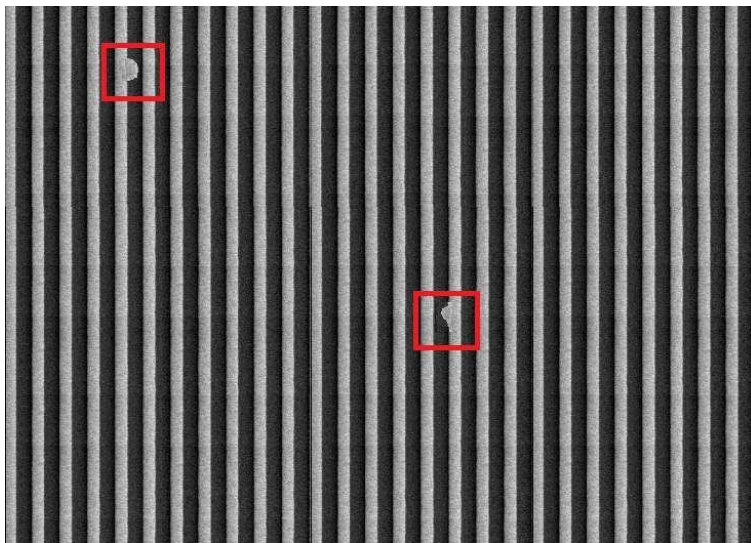
- 웨이퍼 표면 검사: 웨이퍼 표면의 미세 결함, 파티클(이물질), 굽힘 등을 검사



반도체 검사 이미지의 종류

❖반도체 소자 및 패턴 검사 이미지

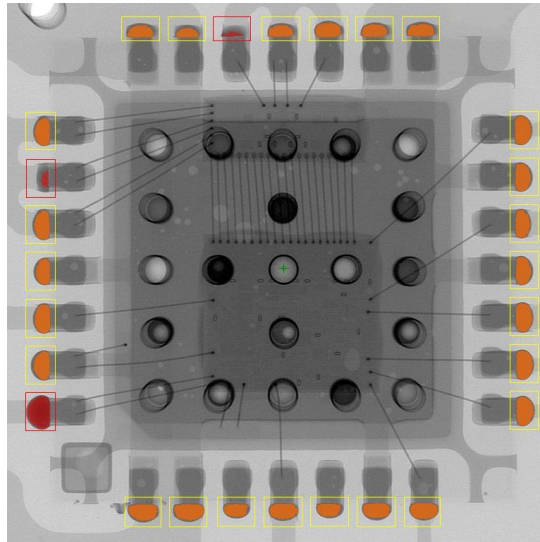
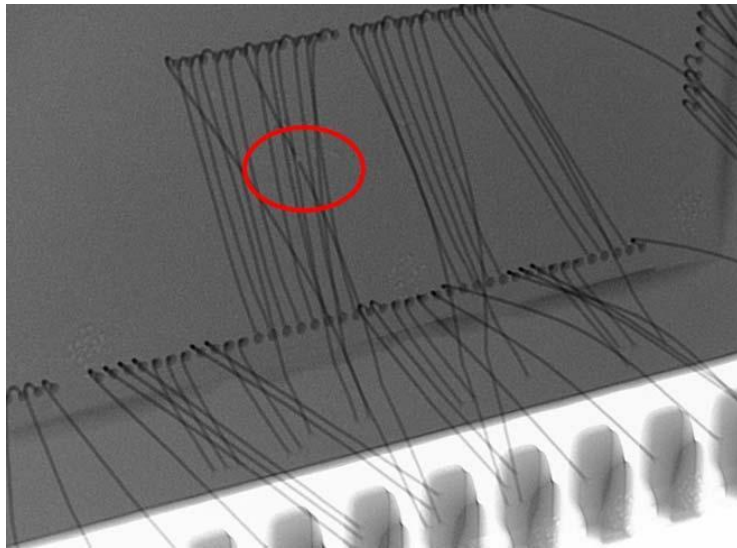
- 회로 패턴이 설계대로 형성되었는지 검사



반도체 검사 이미지의 종류

❖반도체 패키지 및 부품 검사 이미지

- 패키징 후에는 와이어를 눈으로 볼 수 없기 때문에
X-ray 이미지로 단선, 접합 불량, 와이어 솔림(변형) 등을 검사



반도체 검사 이미지와 AI비전

AI비전으로 수행할 수 있는 작업	기술 영역
정상/불량 분류	이미지 분류
결함 유형 분류	이미지 분류
결함 위치 검출	객체 검출
정상 이미지와 다른 패턴 탐색	이상 탐지

본 과목에서 다루는 내용

❖ 인공지능 = 데이터 + 학습

- 데이터 → 반도체 검사 이미지 → 이미지 데이터
- 학습 → AI비전 → 이미지 학습 딥러닝 모델

일정	내용	실습 데이터
1차시~6차시	이미지 데이터의 이해 이미지 학습 딥러닝 모델 데이터 증강	일반적인 실습용 데이터셋 웨이퍼 맵 데이터
6차시	중간고사	
7차시~12차시	이미지 데이터 전처리 및 탐색 웨이퍼 이미지 분류 모델 구현	웨이퍼 촬영 이미지
12차시	기말 프로젝트 발표	

※ 강의 진행 상황에 따라 내용이 변경될 수 있음

2. 딥러닝의 이해

딥러닝 사례 탐구: 딥페이크

- 딥페이크 막는 AI... 디지털 범죄 '쏙쏙' 잡아낸다 / 연합뉴스TV
 - <https://youtu.be/SKkNZ9mvowE?si=qNtcHkyxKHJ3J6VC>



딥러닝 사례 탐구: 주차장 번호판 인식

- '8자리 번호판' 9월 대란설...주차장 90%가 인식 못한다 [출처:중앙일보]
 - <https://www.joongang.co.kr/article/23540176>
- 차량번호 인식 시스템 '신규 번호판 인식' 업데이트
 - <http://www.ssnews.co.kr/news/articleView.html?idxno=10315>

1. 올해 9월 1일 부터 새로운 번호판이 도입됩니다.



2. 차량번호인식시스템이 신규 번호판을 인식할 수 있도록 업데이트해야 합니다.



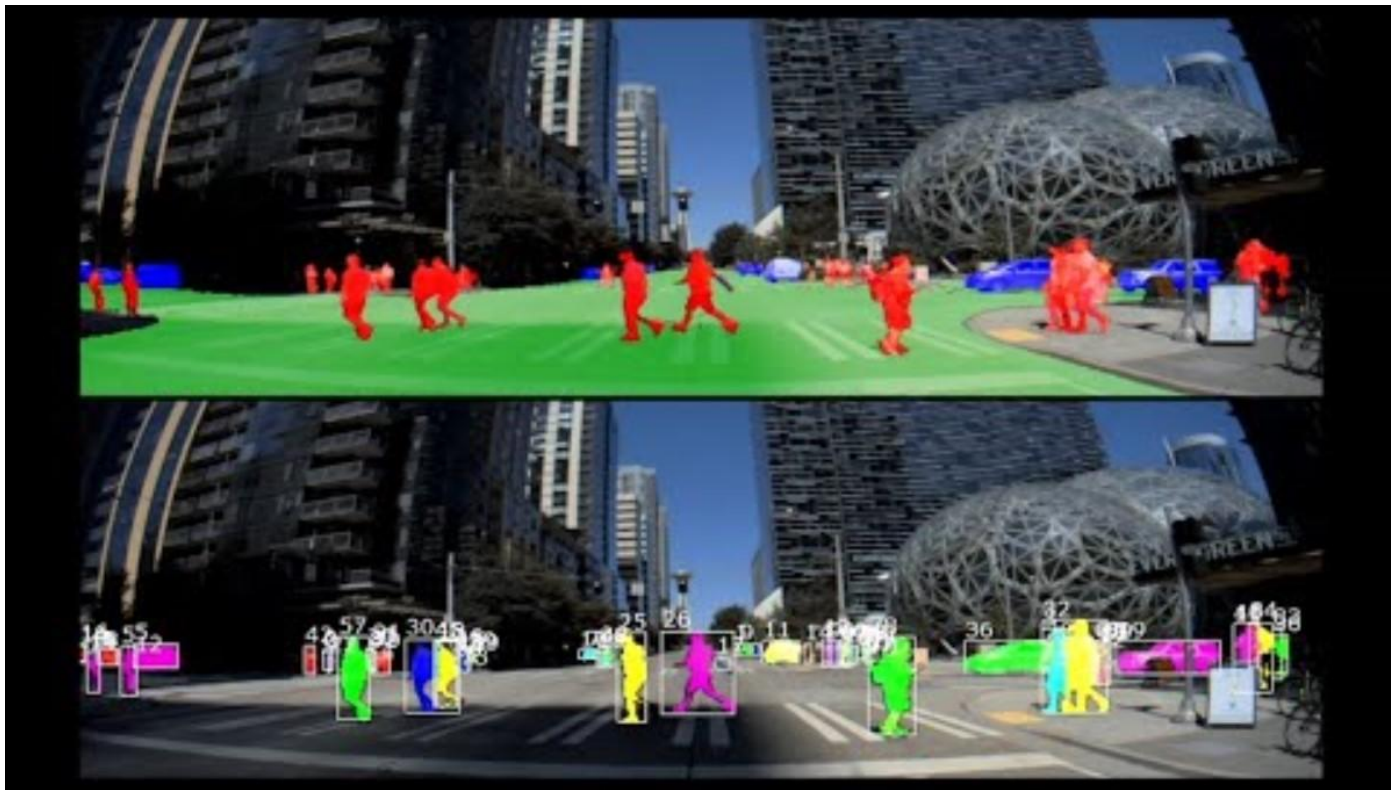
딥러닝 사례 탐구: 음성 인식

■ 해외 주요기업들의 음성인식 AI 에이전트

- 글로벌 IT 기업인 애플, 아마존, MS, 구글 등은 음성 에이전트(비서) AI 플랫폼 출시
- 자사의 음성인식 AI를 기반으로한 시스피커를 자체 판매하거나 제휴를 통해 제품출시

	Apple	Amazon	Microsoft	Google
	 Hey Siri	 alexa	 Cortana Microsoft	 Google Assistant
브랜드	Siri	alexa	Cortana	Google Assistant
출시일	2011년 10월	2014년 11월	2015년 12월	2016년 5월
시스피커 (스마트스 피커)	 홈 팟 홈 팟 미니 (HomePod)	 에코(Echo) 에코 닷 에코 쇼	 하마카돈 인보크	 구글 홈 구글 홈 미니
호출명	"헤이 시리" "시리아"	"알렉사"	"헤이 코타나"	"헤이 구글" "오케이 구글"

딥러닝 사례 탐구: 자율주행 자동차



**머신 러닝과의 차이점,
발견했나요?**

딥러닝에서 달라진 점

- ① 입력의 형태: 비정형 데이터를 다룰 수 있다**
- ② 학습의 형태: 특성 선정과 패턴까지 컴퓨터가 스스로 학습한다**

머신러닝 발전과정에서 주체의 변화

	규칙 기반	데이터 기반 (머신러닝)	딥러닝
입력 데이터	정형 데이터	정형 데이터	비정형 데이터 가능
특성 선정	사람	사람	컴퓨터
패턴 추출(학습)	사람	컴퓨터	컴퓨터

예시: 붓꽃의 품종을 분류하는 문제

부채붓꽃(Iris Setosa)



꽃잎 꽃받침

붓꽃(Iris Versicolor) 북미산 붓꽃(Iris Virginica)



꽃잎 꽃받침



꽃잎 꽃받침

객체
(샘플, 인스턴스, 관측치)

No	꽃받침 길이	꽃받침 너비	꽃잎 길이	꽃잎 너비	종류
1	5.3	3.3	1.3	0.2	부채붓꽃
2	4.8	3.2	1.4	0.3	부채붓꽃
...	...	...	...	...	...
100	7.0	3.6	4.6	1.1	붓꽃
...	...	...	...	...	...
200	6.0	2.8	4.9	1.7	북미산붓꽃

특징
(피처, 속성, 측정값, 설명변수, 독립변수)

라벨
(목적변수, 종속변수)

예시: 붓꽃의 품종을 분류하는 문제

- 규칙 기반



No	특성		정답
	꽃잎길이	꽃잎너비	
1	5.5	2.5	A
2	4.8	3.2	B
⋮	⋮	⋮	⋮

규칙정함

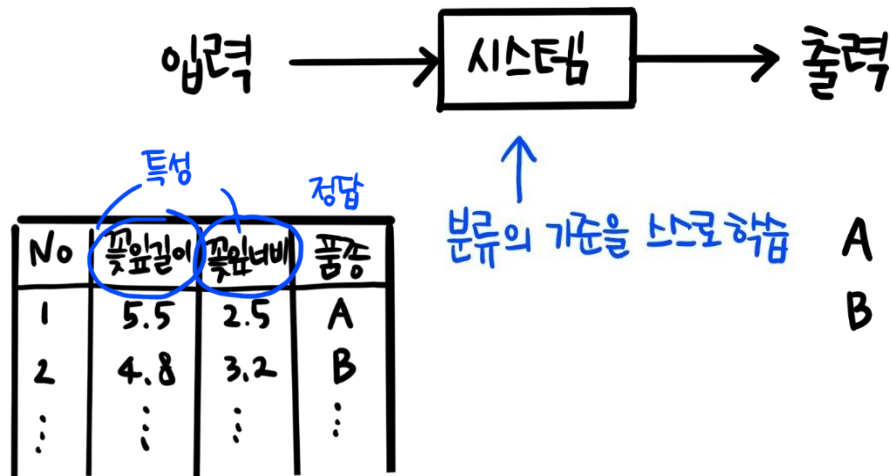
if 꽃잎길이 > 5 A
then A B

if 꽃잎너비 > 3
then B

입력 데이터	이미지를 정형 데이터로 표현해야 함
특성 선정	사람이 특성을 선정하고, 이를 측정 데이터로 만들어 표 형태로 정리 → 새로운 데이터도 동일한 형태로 입력해야 함
패턴 추출(학습)	사람이 규칙을 설정해 해당 조건의 만족 여부에 따라 출력을 결정함

예시: 붓꽃의 품종을 분류하는 문제

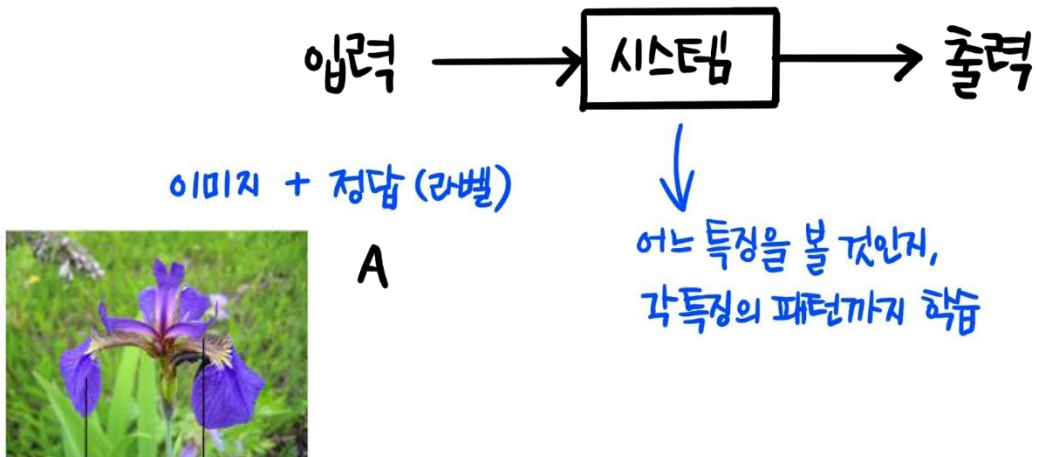
- 데이터 기반



입력 데이터	이미지를 정형 데이터로 표현해야 함
특성 선정	사람이 특성을 선정하고, 이를 측정 데이터로 만들어 표 형태로 정리 → 새로운 데이터도 동일한 형태로 입력해야 함
패턴 추출(학습)	컴퓨터가 데이터의 패턴을 학습하여 분류의 기준을 스스로 학습함

예시: 붓꽃의 품종을 분류하는 문제

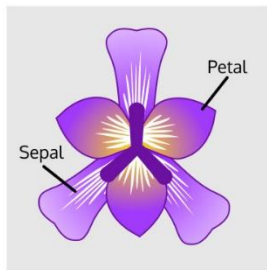
- 딥러닝



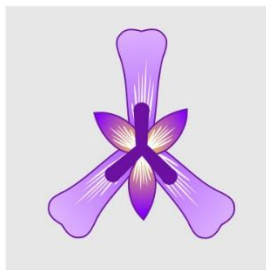
입력 데이터	• 이미지를 그대로 입력할 수 있음 → 새로운 데이터 입력 시에도 이미지로 입력 가능
특성 선정	• 대량의 이미지로부터 어떤 특성을 볼 것인지, 각 특징의 패턴까지 스스로 학습함.
패턴 추출(학습)	

머신 러닝의 한계

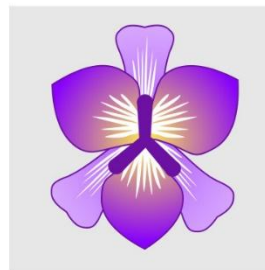
- 붓꽃 품종을 판별하는 머신 러닝 모델을 만든다면 **측정 데이터를 테이블 형태로** 입력해야 함
→ 이렇게 만든 모델로 새로운 붓꽃의 품종을 분류하려면,
꽃받침과 꽃잎의 너비와 길이를 측정한 데이터를 입력해야 함



Iris Versicolor



Iris Setosa



Iris Virginica

- **붓꽃의 이미지만으로 품종을 판별**할 수는 없을까? → 이것 가능하게 하는 게 **딥러닝!**

딥러닝 개념

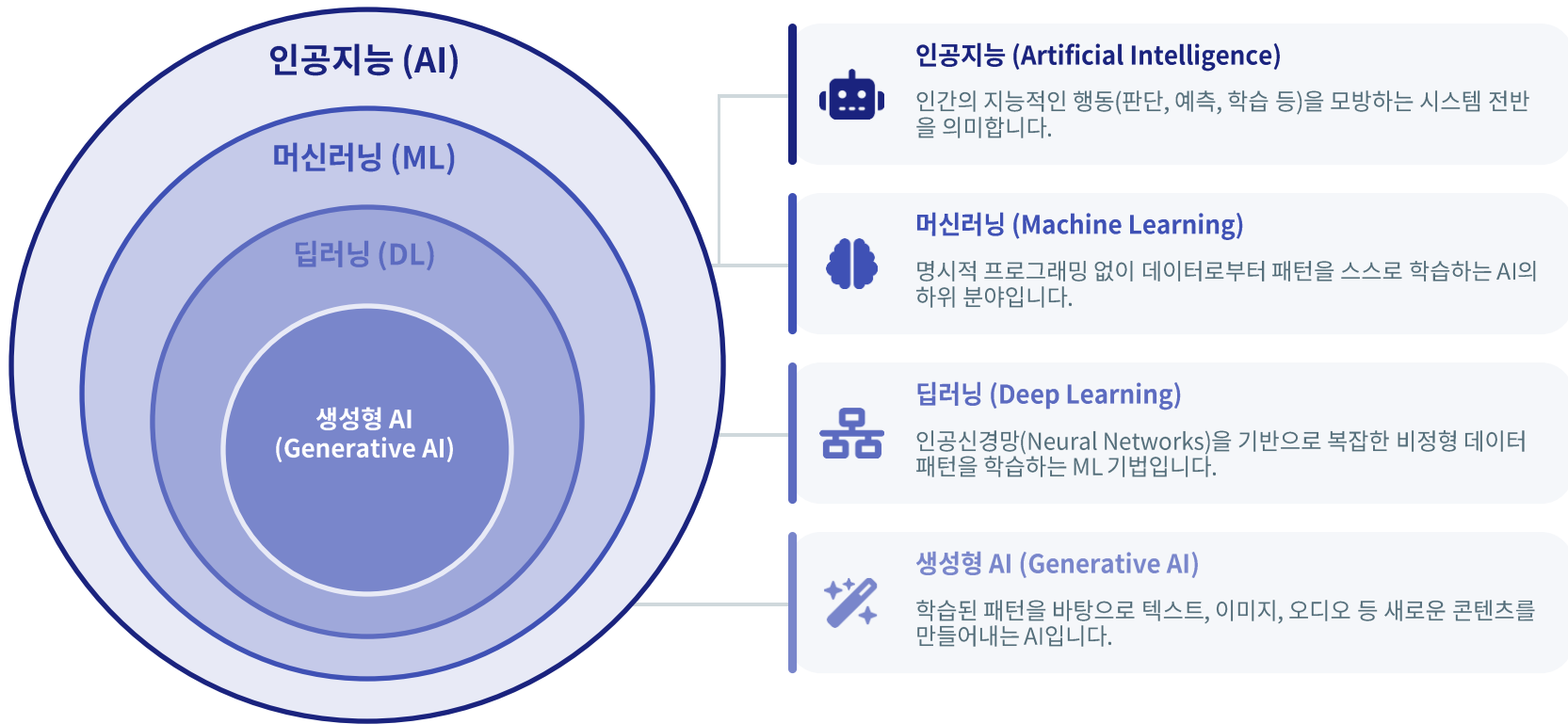
- 모델이 데이터로부터 **직접 특징을 추출하고 학습**하는 머신 러닝 기법
- **비정형 데이터**에 압도적인 강점
 - 스스로 패턴과 특징을 찾아내기 때문에
- 어떻게? **인공신경망**을 깊게 쌓아서
 - 데이터 유형에 따라 다른 인공신경망 모델을 사용
 - 데이터 유형에 따라 다르게 학습하기 때문에



머신러닝과 딥러닝의 차이점

특징	머신러닝 (Machine Learning)	딥러닝 (Deep Learning)
개념	데이터를 기반으로 학습하여 예측/분류하는 알고리즘의 총칭	인공 신경망을 깊게 쌓아 올린 구조를 활용한 머신 러닝의 한 분야
특징 추출	사람이 직접 특징을 정의하고 추출해야 함	모델이 데이터로부터 특징을 자동으로 학습하고 추출함
데이터 유형	주로 정형 데이터에 강점	주로 비정형 데이터에 압도적인 강점
예시	스팸 메일 분류 (키워드/패턴 기반), 주택 가격 예측, 고객 이탈 예측	이미지 인식, 음성 인식, 자연어 번역, 자율주행, 생성형 AI

모든 DL은 ML이지만, 모든 ML이 DL은 아니다.



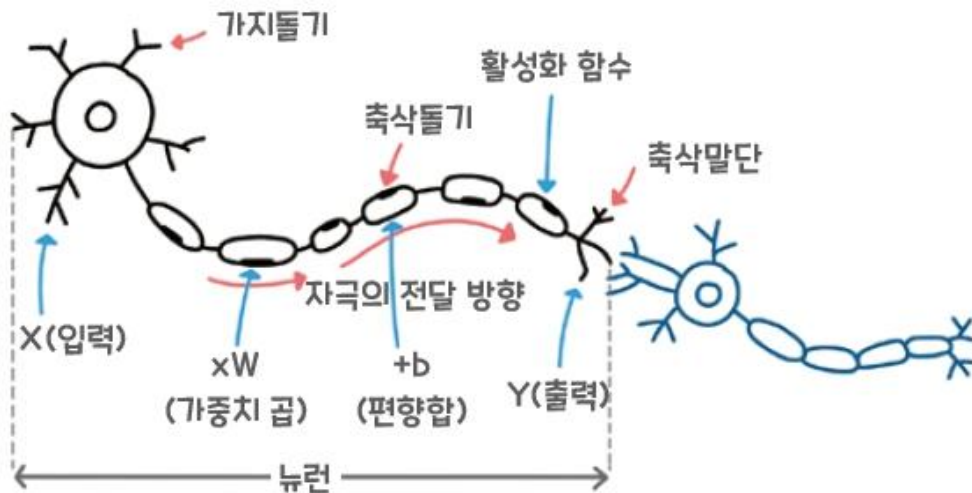
모든 DL은 ML이지만, 모든 ML이 DL은 아니다.



3. 인공지능경망의 이해

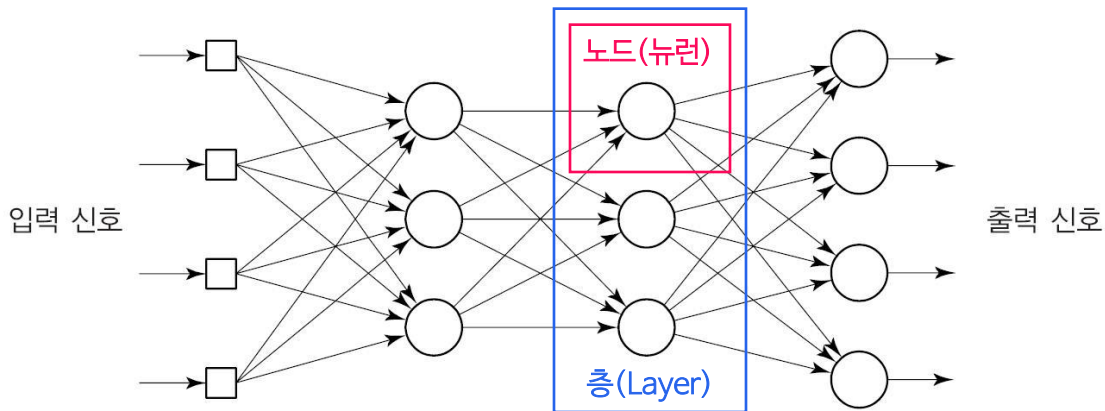
인간의 신경망(Neural Network)

- 뉴런(신경세포)이 시냅스를 통해 연결되어
전기·화학적 신호를 교환하는 뇌의 정보 처리 시스템

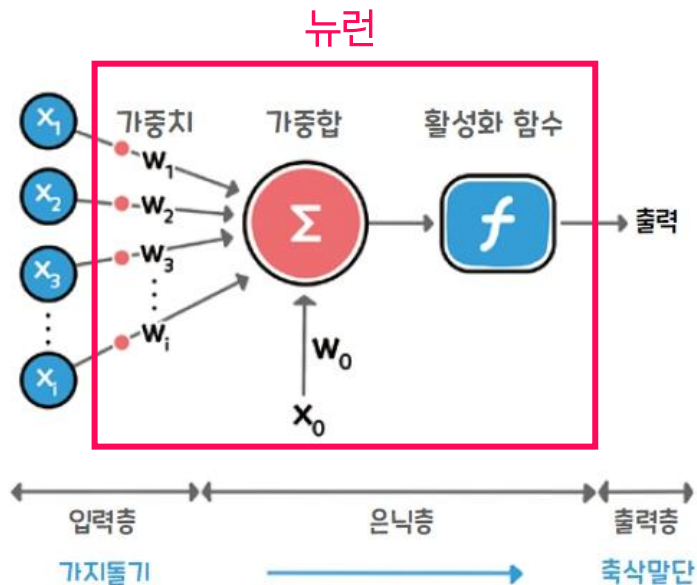


인공신경망(ANN, Artificial Neural Network)

- 정의: 인간 뇌의 뉴런 연결 구조를 모방하는 머신러닝 기법
 - 노드(뉴런) : 정보를 처리하는 최소 단위
 - 층(Layer) : 노드(뉴런)들의 집합 단위



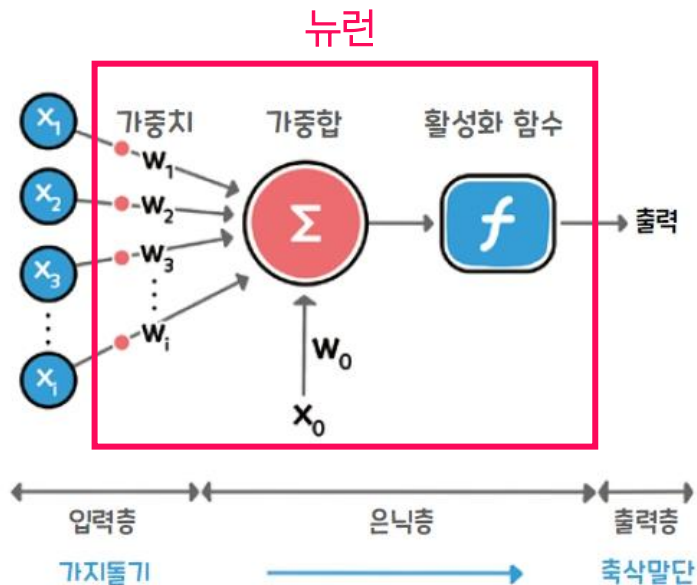
단일 뉴런의 구조



- 인간의 신경망과 대응

- 가지돌기 → 인공신경망의 입력층
- 축삭말단 → 인공신경망의 출력층
- 신호의 크기 → 인공신경망의 가중치

단일 뉴런의 구조



- 주요 개념

- 가중치

- : 입력 신호의 중요도를 조절하는 값
 - : 학습을 통해 업데이트됨.

- 편향

- : 뉴런이 활성화되는 데 필요한 기준값

- 활성화 함수

- : 비선형적인 특징을 신경망에 부여
 - : 신경망이 복잡한 패턴을 학습할 수 있게 함

작동 원리: 가중합(Weighted Sum)

- 입력값과 가중치를 곱한 뒤 편향(Bias)을 더한 값

$$\text{가중합}(y) = (x_0 \times w_0) + (x_1 \times w_1) + (x_2 \times w_2) + b$$

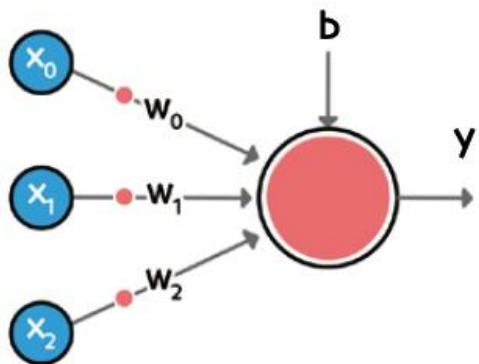
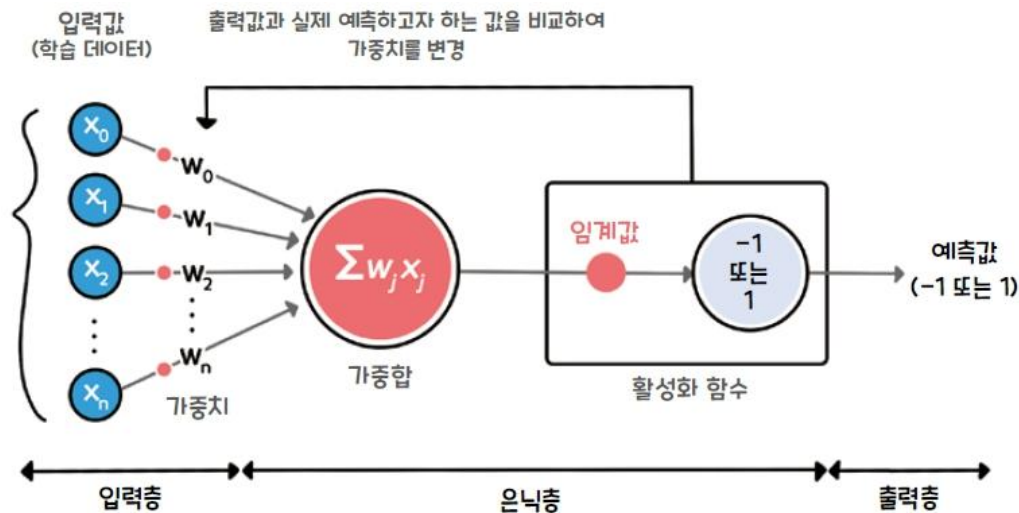


그림 9-10 하나의 노드에 대한 가중합

- 원: 노드
- 화살표: 신호의 흐름
- x_n : 외부에서 들어오는 신호(Input)
- w_n : 신호를 연결하는 가중치(Weight)
- b : 출력값을 조절하는 편향(Bias)
- y : 외부로 나가는 신호(Output)

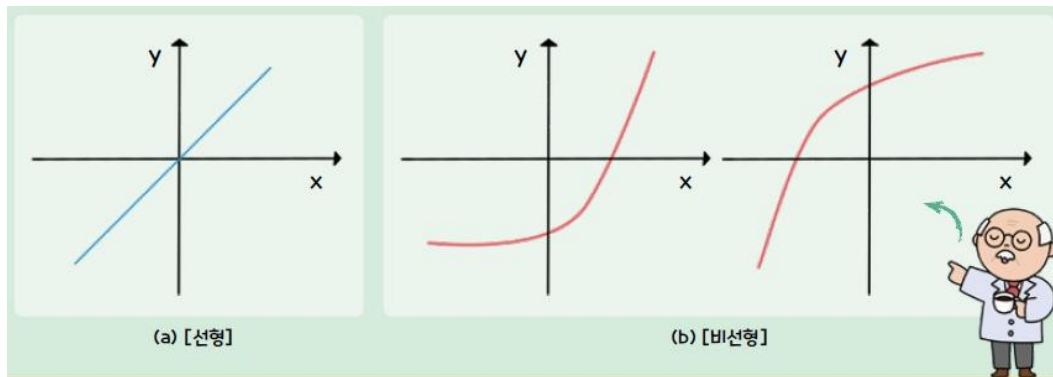
작동 원리: 활성화 함수

- 가중합을 최종 출력값으로 변환하여 다음 층으로 전달하는 비선형 함수
- 비선형 변환을 위한 활성화 함수가 적용돼 복잡한 패턴을 표현할 수 있게 함



일반적인 의미의 선형과 비선형

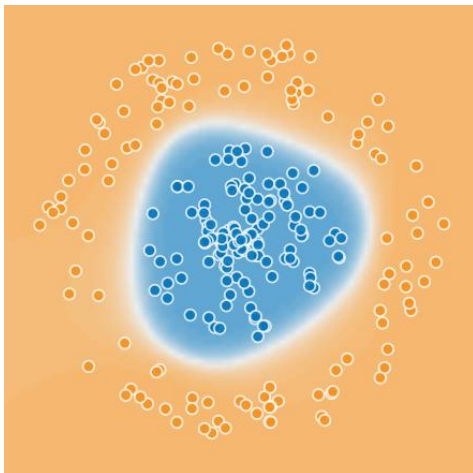
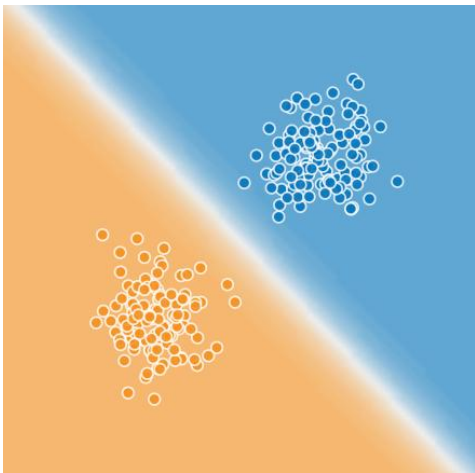
- 선형(Linear)
 - 그림 (a)처럼 그래프가 직선(line)의 모양을 취하는 것으로,
 - 1차 함수의 그래프 모양을 유지하는 것
- 비선형(Non-linear)
 - 직선이 아닌 것으로, 그림 (b)와 같이 그래프에서 곡선 모양을 취하는 것을 비선형이라고 함



머신러닝에서 선형 모델의 한계

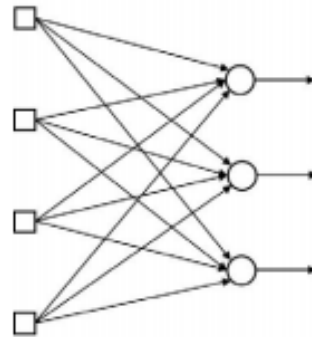
- 복잡한 패턴은 표현이 어렵다.
 - 비선형 경계는 구분이 불가하며 고차원 특징을 추출하기 어렵다.

☞ 직선 경계 vs 곡선 경계



단층 신경망

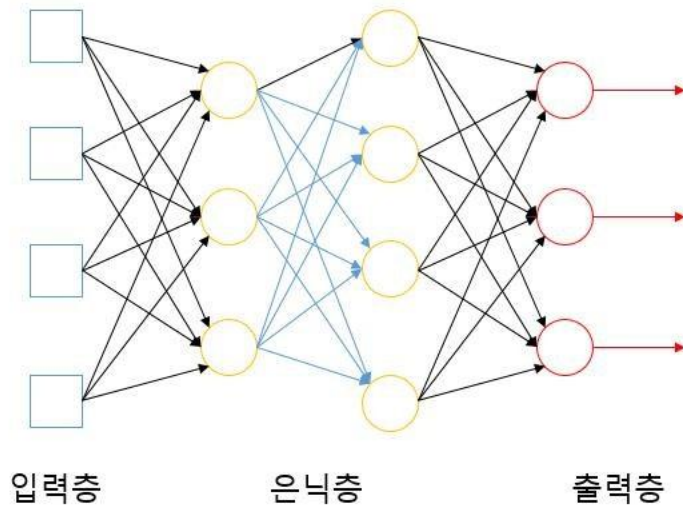
- 단층 신경망
 - 하나의 층(layer)로만 구성된 신경망
≠ 노드(뉴런)가 하나인 것은 아님
 - 하나의 층이란, 입력층 다음에 바로 출력층
 - 동작방식
 - : 여러 개의 입력을 받아 가중합을 구한 후
 - : 활성화 함수를 통과시켜 출력을 내는 모델



단층 신경망

다층신경망 (== 심층신경망)

- 심층신경망(Deep Neural Network, DNN)
- 여러 뉴런이 서로 연결되어 있는 구조의 네트워크
- 입력층과 출력층 사이에 하나 이상의 은닉층(Hidden Layer)을 둔 신경망
 - 입력층은 외부 데이터를 받아들이고,
 - 은닉층은 데이터의 패턴을 학습하며,
 - 출력층은 최종 예측을 생성





TensorFlow Playground 소개

<https://playground.tensorflow.org/>

TensorFlow Playground란?

TensorFlow Playground는 웹 브라우저에서 신경망을 시각적으로 조작하고 관찰할 수 있는 교육용 도구입니다. 데이터셋, 은닉층 수, 학습률, 노이즈 등을 조절하며 신경망의 작동 원리를 직관적으로 확인할 수 있습니다.



데이터셋 선택

4가지 기본 데이터셋 제공



층 구조 조절

은닉층 수/뉴런 수 변경



하이퍼파라미터

학습률, 배치 크기 조절



실시간 시각화

손실/정확도 확인



Epoch
000,216

Learning rate
0.03

Activation
Tanh

Regularization
None

Regularization rate
0

Problem type
Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%



Noise: 0



Batch size: 10



REGENERATE

FEATURES

Which properties do you want to feed in?

X_1



X_2



X_1^2



X_2^2



$X_1 X_2$



$\sin(X_1)$



$\sin(X_2)$



+ - 2 HIDDEN LAYERS

+ -

4 neurons



This is the output from one neuron. Hover to see it larger.

+ -

2 neurons

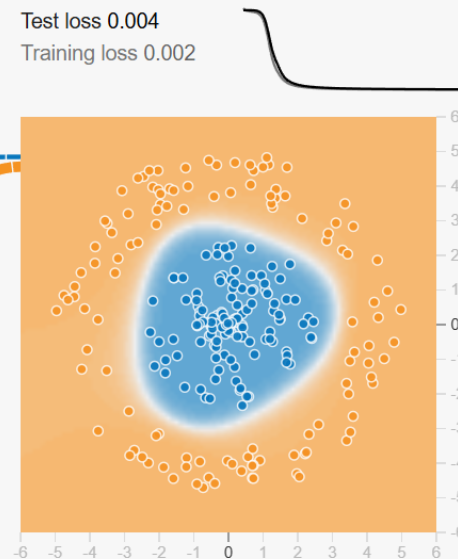


The outputs are mixed with varying weights, shown by the thickness of the lines.

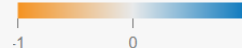
OUTPUT

Test loss 0.004

Training loss 0.002



Colors shows data, neuron and weight values.



① 실습: 단순한 모델

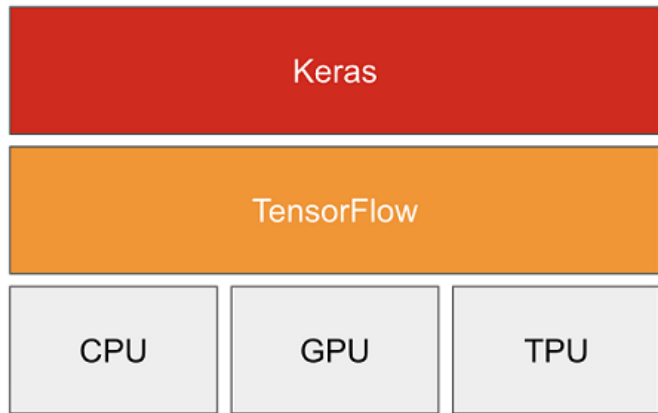
TensorFlow와 Keras

❖TensorFlow(텐서플로우)

- Google에서 개발한 오픈소스 머신러닝·딥러닝 프레임워크
- 신경망 모델의 학습, 연산, 실행 및 배포를 위한 기능 / CPU와 GPU를 활용한 대규모 연산 지원

❖Keras(케라스)

- 딥러닝 모델을 직관적으로 구현하기 위해 TensorFlow 엔진 위에 구축된 고수준API



→TensorFlow가 하드웨어를 제어

→Keras는 TensorFlow를 제어

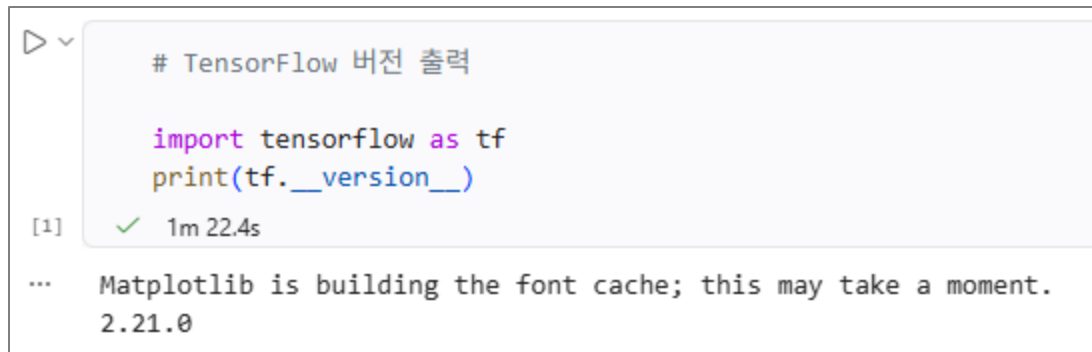
본 과목의 실습 환경 설정하기

❖Google Colab을 사용하는 이유

- 딥러닝 학습에는 많은 연산이 필요하다 → 로컬 컴퓨터에서는 학습이 어려울 수 있음
- 클라우드에서 제공하는 CPU·GPU 등의 컴퓨팅 자원을 활용하기 위해 Colab 사용

❖Colab 확인하기

- Colab에서 다음 코드를 실행 → TensorFlow가 설치되어 있는지 확인
 - 개인 PC에서는 따로 설치해야하지만, Colab은 많이 사용하는 라이브러리가 설치되어 있다.



```
▶ # TensorFlow 버전 출력

import tensorflow as tf
print(tf.__version__)

[1] ✓ 1m 22.4s

... Matplotlib is building the font cache; this may take a moment.
2.21.0
```

실습 예제: 단순한 모델

❖ 다음 코드를 입력하고 실행해보자.

```
import numpy as np
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Dense

x = [-3, 31, -11, 4, 0, 22, -2, -5, -25, -14]
y = [-2, 32, -10, 5, 1, 23, -1, -4, -24, -13]

X_train = np.array(x).reshape(-1, 1)
y_train = np.array(y)

print(X_train.shape, y_train.shape)

model = Sequential()
model.add(Input(shape=(1,)))
model.add(Dense(units=1, activation='linear'))

model.summary()

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.fit(X_train, y_train, epochs=300, verbose=1)

print(model.weights)

model.predict(np.array([[11], [12], [13]]))
```

코드 살펴보기

① 라이브러리 불러오기

```
import numpy as np
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Dense
```

- Sequential : 신경망의 층을 순서대로 쌓아 만드는 모델
- Input : 신경망에 입력되는 데이터의 형태(shape)를 정의
: 모델의 첫 부분에 배치하여 한 개의 데이터가 어떤 구조로 입력되는지 지정
- Dense : 앞 층의 모든 노드와 연결되는 완전연결층

코드 살펴보기

② 학습 데이터 준비

```
x = [-3, 31, -11, 4, 0, 22, -2, -5, -25, -14]
y = [-2, 32, -10, 5, 1, 23, -1, -4, -24, -13]

X_train = np.array(x).reshape(-1, 1)
y_train = np.array(y)

print(X_train.shape, y_train.shape)
```

- 임의의 데이터 생성 : 대략 $y=x+1$ 의 관계를 가진 데이터
- 파이썬 리스트를 Numpy 배열로 변환
 - X_train : 모델에 입력할 특징(feature)
 - y_train : 모델이 예측해야 할 정답(label)
 - reshape(-1, 1) : 10개의 값을 10개 샘플 × 1개 특성 형태로 변환

reshape(-1, 1)

[-3, 31, -11, ...]	[[-3],
	[31],
	[-11],
	...
	[-14]]
(10,)	→ (10, 1)

코드 살펴보기

③ 신경망 모델 만들기

```
model = Sequential()  
model.add(Input(shape=(1,)))  
model.add(Dense(units=1, activation='linear'))  
  
model.summary()
```

- Sequential.add() : 모델에 Layer를 추가함
- Input(shape=(1,)) : 한 개의 샘플이 1개의 특성(feature)을 가짐

```
x_train.shape = (10, 1)
```

↑ ↑
샘플 수 특성 수

```
Input(shape=(1,))
```

↑
샘플 하나의 형태

③ 신경망 모델 만들기

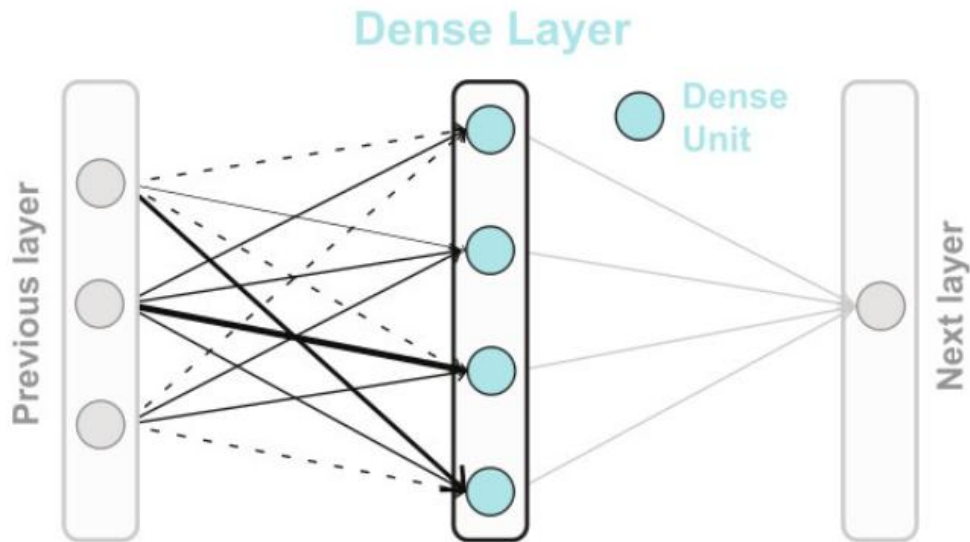
```
model = Sequential()  
model.add(Input(shape=(1,)))  
model.add(Dense(units=1, activation='linear'))  
  
model.summary()
```

- Dense : 신경망의 완전연결층

설정 항목	의미	예제 코드 값
units	층의 출력 노드 수	1
activation	활성화 함수	'linear'

Dense Layer란?

❖ 밀집층 (Dense Layer): 모든 입력이 다음 층의 모든 뉴런과 연결되어 있는 완전연결층



코드 살펴보기

③ 신경망 모델 만들기

```
model = Sequential()  
model.add(Input(shape=(1,)))  
model.add(Dense(units=1, activation='linear'))  
  
model.summary()
```

- `Sequential.summary()` : 생성한 신경망 모델의 구조를 요약하여 출력
: 각 층의 이름, 출력 형태, 학습 파라미터 수 등을 확인

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 1)	2

Total params: 2 (8.00 B)

Trainable params: 2 (8.00 B)

Non-trainable params: 0 (0.00 B)

코드 살펴보기

④ 모델의 학습 방법 설정

```
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

- Sequential.compile() : 모델을 어떤 방법으로 학습하고 평가할지 설정

설정 항목	의미	예제 코드 값
optimizer	오차를 줄이도록 가중치와 편향을 업데이트하는 방법	'adam'
loss	학습 과정에서 최소화해야 할 오차	'mse' (평균제곱오차)
metrics	모델의 성능을 확인하기 위한 평가 지표	['mae'] (평균절대오차)

→ Loss는 모델이 학습할 때 줄여야 하는 값

→ Metric은 사람이 모델의 성능을 확인하기 위한 값

코드 살펴보기

⑤ 모델 학습

```
model.fit(x_train, y_train, epochs=300, verbose=1)

print(model.weights)
```

- Sequential.fit() : 준비한 데이터를 이용하여 실제로 모델을 학습한다.
 - X_train : 입력 데이터
 - y_train : 정답 데이터

설정 항목	의미	예제 코드 값
epochs	전체 데이터셋을 몇 번 반복 학습할 것인지 설정	300
verbose	학습 과정을 화면에 어떻게 출력할 것인지 설정	1 (출력함)

코드 살펴보기

⑥ 새로운 데이터 예측

```
model.predict(np.array([[11], [12], [13]]))
```

- Sequential.predict() : 학습이 끝난 모델에 새로운 입력값을 넣어 결과를 예측한다.

```
1/1 ————— 0s 61ms/step  
array([[ -10.259752],  
       [ -11.215683],  
       [ -12.171615]], dtype=float32)
```

② 실습: 단층 신경망

실습: 패션 아이템 이미지 분류기 만들기

- 패션 아이템 이미지를 입력 받아 어떤 패션 아이템인지 분류하는 모델을 만들어보자.
- 실습 데이터 : Fashion MNIST (딥러닝 학습자를 위해 공개되어 있는 데이터셋)
 - 입력:



- 레이블(라벨/출력):

레이블	0	1	2	3	4	5	6	7	8	9
패션 아이템	티셔츠	바지	스웨터	드레스	코트	샌달	셔츠	스니커즈	가방	앵클 부츠

실습: 단층 신경망 (1)

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense

# 1. Fashion MNIST 데이터 불러오기
(X_train, y_train), (X_test, y_test) = \
    tf.keras.datasets.fashion_mnist.load_data()

print("X_train:", X_train.shape)
print("y_train:", y_train.shape)
print("X_test :", X_test.shape)
print("y_test :", y_test.shape)
```

실행 결과

```
X_train: (60000, 28, 28)
y_train: (60000,)
X_test : (10000, 28, 28)
y_test : (10000,)
```

실습: 단층 신경망 (2)

2. 이미지 데이터 정규화

```
X_train = X_train / 255.0
```

```
X_test = X_test / 255.0
```

- 정규화(Normalization)

- 이미지의 픽셀값을 일정한 범위로 변환하는 전처리
- Fashion MNIST의 픽셀값 0~255를 0~1 범위로 변환
- 입력 데이터의 범위를 일정하게 조정 → 신경망의 학습을 안정적으로 수행할 수 있도록 함

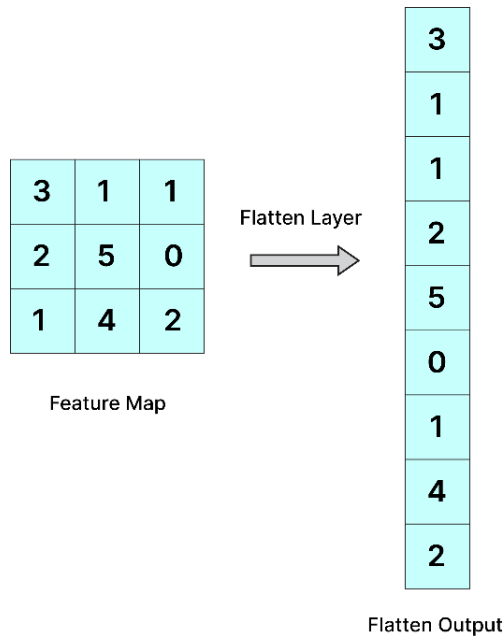
정규화 전		정규화 후
0 ~ 255	→	0.0 ~ 1.0
0		0.0
128	÷ 255 →	0.502
255		1.0

실습: 단층 신경망 (3)

3. 단층 신경망 모델 구성

```
model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),
    Dense(10, activation='softmax')
])
```

- Sequential 모델 생성
 - 입력층(Input)과 평탄화 객체(Flatten),
출력층으로 쓸 레이어 객체(Dense)를 리스트 형태로 전달
- Flatten() : 이전 층의 출력 형태를 받아 1차원으로 펼쳐주는 역할을 함



실습: 단층 신경망 (4)

3. 단층 신경망 모델 구성

```
model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),
    Dense(10, activation='softmax')
])
```

- 출력층: Dense
 - 분류할 항목의 개수가 10개이므로 노드 10개 생성 : 패션 아이템의 레이블은 0~9로 10개 항목
 - 이 레이어의 활성화 함수로 softmax 사용

문제 유형	목표 레이블	활성화 함수
회귀 예측	연속형(숫자형)	선형(linear)
이진 분류	이산형(범주형)	시그모이드(sigmoid)
다중 분류	이산형(범주형)	소프트맥스(softmax)

실습: 단층 신경망 (5)

```
model.summary()
```

summary() 결과

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

28x28 이미지



Input



Flatten

28x28 → 784



Dense(10, softmax)



10개 클래스 예측

실습: 단층 신경망 (6)

4. 모델 학습 방법 설정

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

- 모델 객체를 훈련하기 전에 설정 단계로 최적화 알고리즘, 손실 함수, 성능 지표 등을 설정
- 손실 함수 : 반드시 지정해야 함

문제 유형	정답 형식	손실 함수	출력층 활성화 함수
이진 분류	0 또는 1	binary_crossentropy	sigmoid
다중 클래스 분류	one-hot 레이블	categorical_crossentropy	softmax
다중 클래스 분류	정수형 레이블	sparse_categorical_crossentropy	softmax
회귀	실수	mean_squared_error	없음 또는 linear

실습: 단층 신경망 (7)

```
# 5. 모델 학습
```

```
model.fit(  
    X_train,  
    y_train,  
    epochs=10,  
    batch_size=32,  
    validation_split=0.2  
)
```

- validation_split : 학습데이터의 일부를 검증 데이터로 분리

실습: 단층 신경망 (8)

❖ epoch와 batch size 그리고 iteration

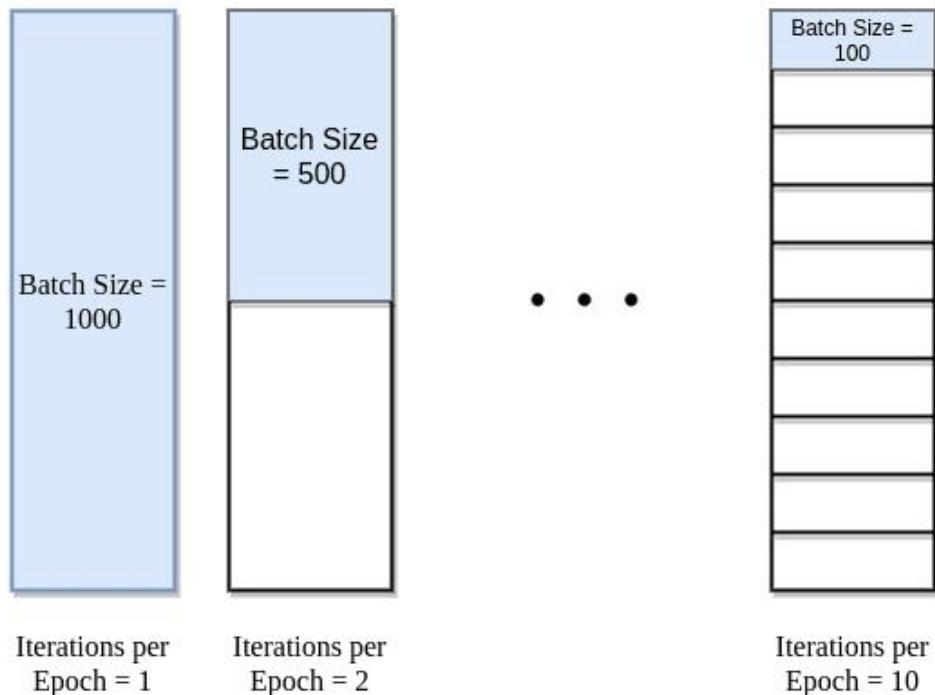


- 1 Epoch : **모든** 데이터 셋을 한 번 학습
1 iteration : 1회 학습

ex) 총 데이터가 100개, batch size가 10이면,
1 iteration = 10개 데이터에 대해서 학습
1 Epoch = $100 / \text{batch size} = 10$ iteration

실습: 단층 신경망 (9)

❖ epoch와 batch size 그리고 iteration



실습: 단층 신경망 (10)

6. 테스트 데이터로 모델 평가

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)
```

```
print("테스트 정확도:", test_accuracy)
```

- Sequential.evaluate() : 학습이 완료된 모델의 성능을 평가하는 함수
 - 학습에 사용하지 않은 테스트 데이터와 정답을 입력
 - compile()에서 지정한 손실함수(loss)와 평가 지표(metrics)의 결과를 반환
 - test_loss : 테스트 데이터의 손실
 - test_accuracy : 테스트 데이터의 정확도

실습: 단층 신경망 (11)

```
# 7. 새로운 이미지 예측
predictions = model.predict(X_test)

print("첫 번째 이미지의 예측값:")
print(predictions[0])
```

- Sequential.predict()
 - 학습이 완료된 모델을 이용하여 새로운 입력 데이터의 결과를 예측하는 함수
 - 입력 데이터 X를 모델에 전달하고 모델의 출력값을 반환
- predict의 결과값을 더 확인하게 좋게 출력하기

```
predicted_class = np.argmax(predictions[0])

print("예측 클래스:", predicted_class)
print("실제 클래스:", y_test[0])
```

③ 실습: 다층 신경망

실습: 다층 신경망 (1)

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense

# 1. Fashion MNIST 데이터 불러오기
(X_train, y_train), (X_test, y_test) = \
    tf.keras.datasets.fashion_mnist.load_data()

print("X_train:", X_train.shape)
print("y_train:", y_train.shape)
print("X_test :", X_test.shape)
print("y_test :", y_test.shape)

# 2. 이미지 데이터 정규화
X_train = X_train / 255.0
X_test = X_test / 255.0
```

실행 결과

```
X_train: (60000, 28, 28)
y_train: (60000,)
X_test : (10000, 28, 28)
y_test : (10000,)
```

실습: 다층 신경망 (2)

3. 다층 신경망 모델 구성

```
model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),
    Dense(128, activation='relu'),    # 은닉층
    Dense(10, activation='softmax')  # 출력층
])
```

- Dense(128, activation='relu') : 128개의 노드를 가진 은닉층
- Dense(10, activation='softmax') : 10개 클래스를 분류하는 출력층
- 활성화 함수
 - ReLU : 0보다 작은 값은 0으로, 0보다 크거나 같은 값은 입력값을 그대로 출력하는 함수
 - softmax : 다중 분류 출력층의 활성화 함수는 반드시 softmax 여야 함

실습: 다층 신경망 (3)

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

Input

(28, 28)



Flatten

28 × 28 → 784



Dense

784 → 128



Dense

128 → 10

→ 단층신경망은 입력값에서 바로 출력 클래스를 결정하지만,

→ 다층신경망은 은닉층에서 입력 데이터의 특징을 조합하고 변환한 뒤 분류한다.

실습: 다층 신경망 (4)

4. 모델 학습 방법 설정

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

5. 모델 학습

```
history = model.fit(  
    X_train,  
    y_train,  
    epochs=10,  
    batch_size=32,  
    validation_split=0.2  
)
```

실습: 다층 신경망 (5)

6. 테스트 데이터로 모델 평가

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)
```

```
print("테스트 정확도:", test_accuracy)
```

```
import numpy as np
```

7. 테스트 데이터 예측

```
predictions = model.predict(X_test)
```

```
predicted_class = np.argmax(predictions[0])
```

```
print("예측 클래스:", predicted_class)
```

```
print("실제 클래스:", y_test[0])
```

끝!